RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Formula Recap:

- **Gini Impurity**:

$$I(D) = 1 - \sum_{i=1}^{n} p_i^2$$

- **Information Gain (using Gini)**:

$$IG(D_p, f) = I(D_p) - \sum_{j=1}^{m} \frac{N_j}{N_p} \cdot I(D_j)$$

Where:

- D_p: Parent dataset
- D_j: Child partition from splitting on feature f
- N_j: Size of child D_j
- N_p: Size of parent dataset

🧮 Step 1: Gini Impurity of the Parent Dataset D_p

From the table, there are:

- Total records $N_p = 14$
- "yes" = 9
- "no" = 5

$$p_{\text{yes}} = \frac{9}{14}, \quad p_{\text{no}} = \frac{5}{14}$$

$$I(D_p) = 1 - \left(\left(\frac{9}{14}\right)^2 + \left(\frac{5}{14}\right)^2\right) = 1 - \left(\frac{81}{196} + \frac{25}{196}\right) = 1 - \frac{106}{196} = \frac{90}{196} \approx 0.459$$

🌳 Step 2: Try Splitting by "age"

"age" has 3 categories: **youth**, **middle_aged**, and **senior**

Let's calculate Gini for each subset.

🔆 age = youth

RID = [1, 2, 8, 9, 11] → 5 instances

- "yes" = 2 (RID 9, 11)
- "no" = 3 (RID 1, 2, 8)

$$p_{\text{yes}} = \frac{2}{5}, \quad p_{\text{no}} = \frac{3}{5} \Rightarrow I(\text{youth}) = 1 - \left(\left(\frac{2}{5}\right)^2 + \left(\frac{3}{5}\right)^2\right) = 1 - \left(\frac{4}{25} + \frac{9}{25}\right) = 1 - \frac{13}{25} = \frac{12}{25} = 0.48$$

🔆 age = middle_aged

RID = [3, 7, 12, 13] → 4 instances

- "yes" = 4
- "no" = 0

  Impurity = 0

☀️ age = senior

RID = [4, 5, 6, 10, 14] → 5 instances

- "yes" = 3 (RID 4, 5, 10)
- "no" = 2 (RID 6, 14)

$$p_{\text{yes}} = \frac{3}{5}, \quad p_{\text{no}} = \frac{2}{5} \Rightarrow I(\text{senior}) = 1 - \left(\frac{9}{25} + \frac{4}{25}\right) = 1 - \frac{13}{25} = \frac{12}{25} = 0.48$$

🧠 Step 3: Weighted Gini after split on "age"

Now calculate:

$$\sum_j \frac{N_j}{N_p} \cdot I(D_j)$$

$$= \frac{5}{14} \cdot 0.48 + \frac{4}{14} \cdot 0 + \frac{5}{14} \cdot 0.48 = \frac{5}{14} \cdot 0.48 + 0 + \frac{5}{14} \cdot 0.48 = \frac{10}{14} \cdot 0.48 = \frac{120}{1400} = 0.343$$

$$IG(D_p, \text{age}) = I(D_p) - \text{Weighted Gini} = 0.459 - 0.343 = \boxed{0.116}$$

2️⃣ Feature: **income**

Categories: high (4), medium (6), low (4)

high:

RID = [1, 2, 3, 13] → 2 yes, 2 no

$$I = 1 - \left(\frac{2}{4}\right)^2 - \left(\frac{2}{4}\right)^2 = 1 - 0.25 - 0.25 = 0.5$$

medium:

RID = [4, 8, 10, 11, 12, 14] → 4 yes, 2 no

$$I = 1 - \left(\frac{4}{6}\right)^2 - \left(\frac{2}{6}\right)^2 = 1 - \frac{16}{36} - \frac{4}{36} = 1 - \frac{20}{36} = 0.444$$

low:

RID = [5, 6, 7, 9] → 3 yes, 1 no

$$I = 1 - \left(\frac{3}{4}\right)^2 - \left(\frac{1}{4}\right)^2 = 1 - 0.5625 - 0.0625 = 0.375$$

Weighted Gini:

$$Gini_{\text{income}} = \frac{4}{14}(0.5) + \frac{6}{14}(0.444) + \frac{4}{14}(0.375) = 0.143 + 0.190 + 0.107 = 0.44$$

$$IG(\text{income}) = 0.459 - 0.44 = \boxed{0.019}$$

3️⃣ Feature: **student**

Categories: yes (7), no (7)

yes:

RID = [5, 6, 7, 9, 10, 11, 13] → 6 yes, 1 no

$$I = 1 - \left(\frac{6}{7}\right)^2 - \left(\frac{1}{7}\right)^2 = 1 - \frac{36}{49} - \frac{1}{49} = 1 - \frac{37}{49} = \frac{12}{49} \approx 0.245$$

no:

RID = [1, 2, 3, 4, 8, 12, 14] → 3 yes, 4 no

$$I = 1 - \left(\frac{3}{7}\right)^2 - \left(\frac{4}{7}\right)^2 = 1 - \frac{9}{49} - \frac{16}{49} = 1 - \frac{25}{49} = \frac{24}{49} \approx 0.489$$

Weighted Gini:

$$Gini_{\text{student}} = \frac{7}{14}(0.245) + \frac{7}{14}(0.489) = 0.1225 + 0.2445 = 0.367$$

$$IG(\text{student}) = 0.459 - 0.367 = \boxed{0.092}$$

4️⃣ Feature: **credit_rating**

Categories: fair (8), excellent (6)

fair:

RID = [1, 3, 4, 5, 8, 9, 10, 13] → 6 yes, 2 no

$$I = 1 - \left(\frac{6}{8}\right)^2 - \left(\frac{2}{8}\right)^2 = 1 - 0.5625 - 0.0625 = 0.375$$

excellent:

RID = [2, 6, 7, 11, 12, 14] → 3 yes, 3 no

$$I = 1 - \left(\frac{3}{6}\right)^2 - \left(\frac{3}{6}\right)^2 = 1 - 0.25 - 0.25 = 0.5$$

Weighted Gini:

$$Gini_{\text{credit}} = \frac{8}{14}(0.375) + \frac{6}{14}(0.5) = 0.214 + 0.214 = 0.428$$

IG(credit_rating) = 0.459 - 0.428 = 0.031

Feature	Gini Index After Split	Information Gain
age	0.343	**0.116**
income	0.440	0.019
student	0.367	0.092
credit_rating	0.428	0.031

$\boxed{\text{age}}$ with $IG = 0.116$

==========

🧠 *What is Gini Index?*

*Gini Index tells us **how mixed the classes are** in a group.*

- *If all examples belong to the same class → Gini = 0 (pure)*
- *If classes are evenly split → Gini is higher (more impure)*

*We use Gini Index to **decide the best feature to split** the data when building a decision tree.*

🤔 *What's the Problem with Gini Index?*

*Sometimes, it **favors features with many unique values**.*

⚠️ *For example:*

Imagine a dataset of customers:

ID	***Age***	***Buys***
1	*25*	*Yes*
2	*30*	*No*
3	*35*	*Yes*
4	*40*	*No*

*Now suppose we split using the **ID**.*

- *Each ID is unique, so every group only has **one row***
- *That means Gini = 0 in each group → Perfect purity!*
- *So Gini Index would say **"Yes! Split by ID!"***

*But... that's **not useful at all**, right? We're not learning anything general. We're just memorizing.*

✅ That's Why We Use ~~Gini~~ Gain Ratio [A Short Story]

~~Gini~~ Gain Ratio = Gini Gain / Split Information

- ***Gini Gain****: how much impurity is reduced*
- ***Split Information****: how many branches the feature creates and how balanced they are*

🛠️ ~~Gini~~ Gain Ratio fixes this by penalizing splits that:

- *Create **too many branches***
- *Are **too unbalanced***

*It prevents the model from picking useless features like **ID** or **Name** just because they happen to make "pure" groups.*

🎯 Real Example

Let's say we have two features:

1. ***"Age"****: has 3 values → (young, middle-aged, senior)*
2. ***"Customer ID"****: 100 different values (each customer has their own)*

Gini Index might say:

> *"Customer ID" is best because every split has pure labels*

But:

- *"Customer ID" creates 100 tiny branches*
- *It doesn't help us generalize or learn anything useful*

~~Gini~~ Gain Ratio steps in and says:

> *"Whoa! Too many branches! You're overfitting!"*

*So it reduces the **score** for Customer ID and might pick **"Age"** instead, which is **more meaningful**.*

✅ *In short:*

Concept	***Description***
Gini Index	*Measures how pure or impure a group is after a split*
Gain ***~~Gini~~ Ratio***	*Adjusts Gini Index to avoid splitting on too many or unhelpful categories*

*It's like having a **referee** that says:*

> *"Yes, you got pure groups — but did you cheat by splitting too much?"*

Gain

🧠 What is the ~~Gini~~ Ratio?

While the **Gini Index** measures the impurity of a split, the **Gini Ratio** helps prevent a bias toward attributes with many distinct values (especially categorical variables with many unique values, like ID or name).

Gain

$$\text{~~Gini~~ Ratio}(f) = \frac{IG(D_p, f)}{SplitInfo(f)}$$

Where:

- $IG(D_p, f)$: Gini-based Information Gain (already computed)
- $SplitInfo(f) = -\sum_{j=1}^{m} \frac{N_j}{N_p} \log_2 \left(\frac{N_j}{N_p}\right)$

✳️ Step-by-Step for Each Feature

Let's begin with **"age"**.

1️⃣ Feature: age

Categories: youth (5), middle_aged (4), senior (5)

Total = 14

Proportions:

- youth = $\frac{5}{14}$
- middle_aged = $\frac{4}{14}$
- senior = $\frac{5}{14}$

$$SplitInfo(\text{age}) = -\left(\frac{5}{14}\log_2\frac{5}{14} + \frac{4}{14}\log_2\frac{4}{14} + \frac{5}{14}\log_2\frac{5}{14}\right)$$

$$= -\left(2 \cdot \frac{5}{14} \cdot \log_2\frac{5}{14} + \frac{4}{14} \cdot \log_2\frac{4}{14}\right)$$

Now, compute values:

- $\frac{5}{14} \approx 0.357 \Rightarrow \log_2(0.357) \approx -1.485$
- $\frac{4}{14} \approx 0.286 \Rightarrow \log_2(0.286) \approx -1.807$

$$SplitInfo = -\left(2 \cdot 0.357 \cdot (-1.485) + 0.286 \cdot (-1.807)\right) = -\left(-1.059 + (-0.517)\right) = 1.576$$

$$\text{~~Gini~~ Ratio}_{\text{age}} = \frac{0.116}{1.576} \approx \boxed{0.074}$$

Gain

2️⃣ Feature: income

Categories: high (4), medium (6), low (4)

Proportions:

- high = 4/14
- medium = 6/14
- low = 4/14

$$SplitInfo(\text{income}) = -\left(\frac{4}{14}\log_2\frac{4}{14} + \frac{6}{14}\log_2\frac{6}{14} + \frac{4}{14}\log_2\frac{4}{14}\right)$$
$$= -\left(2\cdot\frac{4}{14}\cdot\log_2\frac{4}{14} + \frac{6}{14}\cdot\log_2\frac{6}{14}\right)$$

- $\frac{4}{14} \approx 0.286 \Rightarrow \log_2(0.286) \approx -1.807$
- $\frac{6}{14} \approx 0.429 \Rightarrow \log_2(0.429) \approx -1.222$

$$SplitInfo = -\left(2\cdot 0.286\cdot -1.807 + 0.429\cdot -1.222\right) = -\left(-1.033 + (-0.524)\right) = 1.557$$

$$\cancel{\text{Gini}}\ \text{Ratio}_{\text{income}} = \frac{0.019}{1.557} \approx \boxed{0.012}$$

Gain

3️⃣ Feature: student

Values: yes (7), no (7)

$$SplitInfo(\text{student}) = -2\cdot\frac{7}{14}\cdot\log_2\left(\frac{7}{14}\right) = -2\cdot 0.5\cdot\log_2(0.5) = -1\cdot(-1) = 1$$

$$\cancel{\text{Gini}}\ \text{Ratio}_{\text{student}} = \frac{0.092}{1} = \boxed{0.092}$$

Gain

4️⃣ Feature: credit_rating

fair (8), excellent (6)

- $\frac{8}{14} \approx 0.571 \Rightarrow \log_2(0.571) \approx -0.807$
- $\frac{6}{14} \approx 0.429 \Rightarrow \log_2(0.429) \approx -1.222$

$$SplitInfo = -\left(0.571\cdot -0.807 + 0.429\cdot -1.222\right) = -(-0.461 + -0.524) = 0.985$$

$$\cancel{\text{Gini}}\ \text{Ratio}_{\text{credit_rating}} = \frac{0.031}{0.985} \approx \boxed{0.031}$$

Gain

Feature	Gini Gain	SplitInfo	Gini Ratio
age	0.116	1.576	**0.074**
income	0.019	1.557	0.012
student	0.092	1.000	**0.092**
credit_rating	0.031	0.985	0.031

🔥 Best Feature by Gini Ratio:

$$\boxed{\text{student}} \quad (\text{highest Gini Ratio} = 0.092)$$

Even though **"age"** gave the best raw Gini Gain, the **Gini Ratio** suggests **"student"** is the better split (more balanced, less bias from multiple values).

*REMEMBER **BY DEFAULT** IF NOT ASKED SPECIFICALLY -* ***DECISION TREE INFORMATION GAIN*** *WORKS* ***BASED ON Entropy Impurity***

Splitting Method
—----------
*The **Minimum Description Length (MDL)** principle is all about finding the model that **compresses [Encoding]** the data best — the shortest possible description of both the model and the data it explains.*

Tree Pruning

1. *Noise/ Outlier -> Tree Can be Infinite Depth*
2. *Tree Pruning -> Prevent Overfitting*
    a. *Smaller, and Less Complex Tree*
3. *Pruning*
    a. *Prepruning*
        i. *not to further split the tree*
        ii. *stop it early*
        iii. *Too Early: Underfit*
        iv. *Too Late: Overfit*
    b. *Postpruning*
        i. *Fully Grown*
        ii. *Replacing with Leaf [Majority]*
4. *CART => Error Rate (Cost Complexity Pruning), and C 4.5 (Pesimistic Pruning)*

1. Repetition

Repetition happens when the same attribute is repeatedly tested along the same branch of the decision tree.

Example: Suppose we have a decision tree where the first question asks if a person is younger than 60. Then, in the next step, it asks the same question again: "Is age less than 60?" over and over. This repetition makes the tree redundant and harder to follow.

```
Is age < 60?
   Yes → Is age < 45?
       Yes → Is age < 35?
           Yes → Class A
           No → Class B
       No → Class C
   No → Class D
```

You can see the question "Is age < 60?" keeps repeating itself multiple times along the same branch, which doesn't make sense and is confusing.

2. Replication

Replication happens when the same subtree (a branch of questions) appears in different places in the tree, creating redundancy.

Example: Let's imagine a decision tree where the same set of questions appears in two places. For instance, there could be a subtree that asks about "credit rating" and then "income" under multiple branches in the tree.

```
Is age < 60?
   Yes → Is credit rating good?
       Yes → Is income high?
           Yes → Class A
           No → Class B
       No → Is credit rating good?
           Yes → Is income high?
               Yes → Class A
               No → Class B
           No → Class C
   No → Class D
```

In this case, the same subtree (about "credit rating" and "income") appears twice, creating unnecessary duplication, which makes the tree more complex than it needs to be.

3. Multivariate Splits

To avoid **repetition** and **replication**, we can use **multivariate splits**, where we combine attributes into one split instead of asking about one attribute at a time.

Example: Instead of asking "Is age < 60?" repeatedly or asking about the same questions multiple times (like credit rating and income), we can combine multiple attributes into a single split.

For instance, instead of creating multiple branches for "Is age < 60?" and "Is credit rating good?", we can combine age and credit rating into one question:

```
Is age < 60 AND credit rating good?
   Yes → Is income high?
       Yes → Class A
       No → Class B
   No → Class C
```

Now, we've combined age and credit rating into a single question, and there's no repetition or replication, making the tree simpler and easier to follow.

What does AVC do in RainForest as an improvement?

AVC = Attribute-Value, Classlabel.

RainForest uses **AVC-sets** to make decision tree construction more **scalable** (i.e., it works better with huge data sets that don't fit in memory).

✅ What does AVC do?

At each node in the tree, for each attribute, RainForest stores a summary called an **AVC-set**. This tells us:

Table 8.1 Class-Labeled Training Tuples from the *AllElectronics* Customer Database

RID	*age*	*income*	*student*	*credit_rating*	*Class: buys_computer*
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

	buys_computer	
age	yes	no
youth	2	3
middle_aged	4	0
senior	3	2

	buys_computer	
income	yes	no
low	3	1
medium	4	2
high	2	2

	buys_computer	
student	yes	no
yes	6	1
no	3	4

	buys_computer	
credit_ratting	yes	no
fair	6	2
excellent	3	3

Figure 8.8 The use of data structures to hold aggregate information regarding the training data (e.g., these AVC-sets describing Table 8.1's data) are one approach to improving the scalability of decision tree induction.

➡️ *For each value of the attribute, how many tuples (rows) have each class label.*

For example:

```
Attribute: age
Value: youth -> yes: 2, no: 3
        senior -> yes: 4, no: 0
```

This means we don't have to go through the entire data set over and over—we just use these small AVC-sets to decide how to split the data.

✅ **Why is it an improvement?**

- It **reduces memory usage** (only keeps small summaries instead of full data).
- Helps the algorithm **avoid loading the entire dataset into memory**.
- Makes the algorithm work even on **millions of rows**.

In BOAT, why are only 2 scans needed?

BOAT (Bootstrapped Optimistic Algorithm for Tree construction) handles large data **without storing special data structures** like AVC-sets.

✅ Why 2 scans?

BOAT works in **two main steps**:

1. **First Scan**:
    - It draws several **samples** from the full dataset that can fit in memory.
    - It builds several **small trees** using those samples.
    - Then it combines them into a "good guess" tree called **T_0**.
2. **Second Scan**:
    - It checks if T_0 is *close enough* to the true tree.
    - If not, it **refines T_0** using information from the full data.

✅ Why only 2 scans is a big deal?

Most decision tree algorithms need **one scan per tree level**, which adds up quickly.
BOAT only scans the data **twice total**, making it **much faster and more scalable**.

What does the visualization (PBC) help with?

PBC (Perception-Based Classification) is about **interactively building** a decision tree using **visual tools**.

✅ **How does it help?**

- **Visual data exploration**: You can see patterns in data using colors and shapes.
- **Manual splitting**: You choose attributes and where to split based on your knowledge.
- **Better understanding**: You understand your data deeply while interacting with it.
- **Smaller, simpler trees**: The trees built are often more compact and easier to interpret.

Imagine looking at a circle with colored segments—each segment represents an attribute, and color shows the class. You spot patterns like “when income is high, most people don’t buy a computer,” and you can **decide to split** based on that.

✅ **Big advantage**: You combine **human intuition** with machine learning, leading to smart, interpretable models.

TL;DR

Feature	Key Benefit	What it Solves
RainForest AVC	Keeps summaries instead of full data	Saves memory, improves scalability
BOAT (2 scans)	Uses sampling & tree merging	Fast training on huge data
PBC Visualization	Lets user guide tree building	Better understanding, smaller trees

Summary of the Scenarios:

Scenario	Condition	Action
1	All tuples belong to the same class	Convert node to a leaf and label it with that class.
2	No remaining attributes to split	Use majority voting to label the node with the most common class.
3	A branch (partition) is empty	Label the empty branch with the majority class of the parent node.

Algorithm	Attribute Selection Measure	Typical Splits
ID3	Information Gain (Entropy)	Multiway splits
CART	Gini Index or Classification Error	Binary splits
C4.5	Gain Ratio (modified Information Gain)	Multiway splits

Calculating Information Gain for a Continuous Attribute: A Step-by-Step Guide

In decision tree algorithms, **Information Gain (IG)** is used to determine the best feature and split point to partition the data in a way that reduces uncertainty. When dealing with continuous attributes, the process involves choosing the best split based on midpoints between adjacent values. In this example, we will compute the Information Gain for a continuous attribute, "Age," to demonstrate how the concept works.

Steps to Compute Information Gain for a Continuous Attribute:

1. **Sort the Data by the Continuous Attribute**:
    - First, the data should be sorted by the continuous attribute. In this case, "Age."
2. **Consider Midpoints Between Adjacent Values**:
    - The next step is to identify potential split points by calculating midpoints between adjacent values in the sorted data.
3. **Create Two Partitions**:
    - For each midpoint, divide the dataset into two partitions: one with values less than or equal to the midpoint, and one with values greater than the midpoint.
4. **Evaluate the Information Gain for Each Split**:
    - Calculate the entropy of each partition and the overall dataset before and after the split, and use this to compute Information Gain.
5. **Find the Best Split Point**:
    - The split that minimizes entropy (or maximizes Information Gain) is considered the best split.

Example Walkthrough:

Given Data:

Index	Age	Class
1	22	Yes
2	25	No
3	27	Yes
4	30	No
5	32	Yes

Step 1: Sort the Data by Age

The data is already sorted by the "Age" attribute:
[22, 25, 27, 30, 32]

Step 2: Consider Midpoints Between Adjacent Values
The midpoints between adjacent "Age" values are calculated as:

- Between 22 and 25: (22 + 25) / 2 = 23.5
- Between 25 and 27: (25 + 27) / 2 = 26
- Between 27 and 30: (27 + 30) / 2 = 28.5
- Between 30 and 32: (30 + 32) / 2 = 31

Step 3: Create Partitions for Each Split Point

- **Split at 23.5:**
  - D1 = [22] (Class: Yes)
  - D2 = [25, 27, 30, 32] (Classes: No, Yes, No, Yes)
- **Split at 26:**
  - D1 = [22, 25] (Classes: Yes, No)
  - D2 = [27, 30, 32] (Classes: Yes, No, Yes)
- **Split at 28.5:**
  - D1 = [22, 25, 27] (Classes: Yes, No, Yes)
  - D2 = [30, 32] (Classes: No, Yes)
- **Split at 31:**
  - D1 = [22, 25, 27, 30] (Classes: Yes, No, Yes, No)
  - D2 = [32] (Class: Yes)

Step 4: Calculate the Entropy for Each Partition

- **Parent Node (I(D))**:
  - "Yes" = 3 instances
  - "No" = 2 instances
  - Total instances (Np) = 5
  - Entropy of the parent node:
    I(D)=−(3/5log2(3/5)-2/5log2(2/5))

    = ……..

- **Partition 1 (D1)** for Split at 23.5:
  - D1 = [22] (Class: Yes)
  - Entropy of D1: 0

    I(D1) = ……..

- **Partition 2 (D2)** for Split at 23.5:
  - D2 = [25, 27, 30, 32] (Classes: No, Yes, No, Yes)
  - "Yes" = 2 instances
  - "No" = 2 instances
  - Entropy of D2:
    I(D2)=−(2/4log2(2/4)-2/4log2(2/4))

    I(D2) = ……..

Step 5: Calculate the Information Gain

- **Information Gain for Split at 23.5**:

  IG(Dp,f)= I(D)−(N1/Np×I(D1)+N2/Np×I(D2))

  = ……..

==========

Assume:

- You have **n = 10 attributes** (like Age, Income, Gender, etc.)
- Your dataset D has **|D| = 1,000 tuples** (or rows)

🔍 Step-by-step Explanation:

1. **At each node** of the decision tree, the algorithm:
   - Evaluates all **n attributes**
   - For each attribute, it needs to check the best way to split the data (which can involve sorting, especially for numerical features)
   - Sorting or checking for best split often takes log(|D|) time
2. Since it needs to do this for **each tuple**, and for **each attribute**, and potentially **at every level** of the tree, the overall time is approximately:

O(n×|D|×log(|D|))=O(10×1000×log(1000))That's:

O(10×1000×10)=O(100,000)

So roughly **100,000 operations** to build the tree.

FOCUS ON …

1. Splitting* [Entropy, GINI, GINI RATIO]
2. PRUNING*
3. IMPROVEMENTS [RAINFOREST, BOAT]
4. CONTINUE … [Next Topic: Naive Bayes]